

Sujet de Projet : Reproduction du jeu *Pac-Man* avec Unity, C# et Outils de Gestion de Projet

Contexte du projet

L'objectif de ce projet est de **reproduire une version simplifiée du célèbre jeu *Pac-Man***. Ce travail servira à consolider les compétences en **programmation orientée objet (C#)**, en **conception de jeux avec Unity**, et en **gestion de projet logiciel**. Les étudiants devront concevoir le gameplay, les comportements des fantômes, les collisions, le score et l'interface, tout en appliquant une méthodologie de travail organisée à l'aide d'outils professionnels.

 Pac-Man, la vidéo explicative

Objectifs pédagogiques

♦ **Techniques**

- Appliquer les principes de **programmation orientée objet en C#**.
- Utiliser **Unity** pour gérer la logique de jeu, les collisions, les sprites et l'interface utilisateur.
- Comprendre et implémenter les **mécaniques de base de Pac-Man** : déplacement, collecte, poursuite et détection de collision.

♦ **Gestion de projet**

- Utiliser **Git** pour le contrôle de version et la collaboration.
 - Organiser les tâches avec **Trello** (gestion de backlog, sprints, état d'avancement).
 - Planifier les différentes étapes du projet à l'aide d'un **diagramme de Gantt**.
 - Produire une **analyse UML complète** (diagrammes de classes, d'activités, d'états et de cas d'utilisation) liste de diagramme non exhaustive.
-

Fonctionnalités attendues

1. Déplacement du Joueur (Pac-Man)

- Mouvement fluide à l'aide des touches directionnelles (haut, bas, gauche, droite, ou autre).
- Gestion des collisions avec les murs du labyrinthe.
- Animation simple du personnage (ou alternance de sprites pour simuler la bouche qui s'ouvre et se ferme).

2. Labyrinthe

- Construction du plateau de jeu à partir d'un **Tilemap** ou de blocs manuels.
- Les murs empêchent le passage du joueur et des fantômes.
- Des **pastilles** (points) disposées dans les couloirs à collecter.

3. Fantômes (IA basique)

- Les fantômes se déplacent automatiquement dans le labyrinthe.
- Comportement simple (déplacement aléatoire ou semi-aléatoire).
 - Attention voir pour 2 à 4 comportement prédéfini (en fonction de la couleur)
 - possible d'imaginer des comportements
 - possible de changer les comportements de base
- Collision avec le joueur → Game Over.
- Ajout d'une "super pastille" permettant à Pac-Man de manger les fantômes pendant quelques secondes.

4. Score et Interface

- Le score augmente à chaque pastille collectée.
- Interface affichant le score et les vies.
- Écran de Game Over avec option de recommencer.

5. Gestion de Niveaux et Difficulté

- Possibilité d'augmenter la vitesse des fantômes au fil du jeu.
- Option bonus : ajout d'une "super pastille" permettant à Pac-Man de manger les fantômes pendant quelques secondes.

6. IA qui joue PACMAN

- Création d'une IA capable de jouer au jeu en s'inspirant de la vidéo suivante :
 J'ai codé un robot qui DÉTRUIT Pacman.

Analyse UML Obligatoire

Chaque groupe devra produire une **analyse UML complète** :

- **Diagramme de cas d'utilisation** : Interaction entre le joueur et les éléments du jeu (déplacer Pac-Man, collecter pastilles, éviter fantômes...).
- **Diagramme de classes** : Structure des principales entités (Joueur, Fantôme, Grille, Pastille, ScoreManager...).
- **Diagramme d'activités** : Séquence logique d'un tour de jeu (mouvement → collision → mise à jour du score).
- **Diagramme d'états** : États possibles de Pac-Man (en jeu, en collision, invincible, game over).

Outils et contraintes techniques

Outil / Langage	Utilisation
Unity (2D)	Moteur de jeu
C#	Programmation du gameplay
Git / GitHub / GitLab	Versionning du code
Trello	Organisation du travail en tâches et suivi d'avancement
Diagramme de Gantt	Planification du projet
UML (Lucidchart, Draw.io, ou autre)	Conception du système avant le développement

 Phases de développement**Phase 1 – Analyse et conception**

- Définition du périmètre fonctionnel du jeu.
- Création des diagrammes UML.
- Mise en place du Trello et du Gantt.
- Initialisation du dépôt Git et du projet Unity.

Phase 2 – Développement du prototype

- Implémentation des mouvements et des collisions.
- Création du labyrinthe et des pastilles.
- Ajout du système de score.

Phase 3 – IA et finitions

- Ajout des fantômes et de leur comportement.
- Ajout de l'écran de Game Over et du redémarrage.
- Test et ajustements (debug, optimisation).

Phase 4 – Documentation et présentation

- Rédaction du rapport de projet (avec UML, captures de Trello et du Gantt).
 - Présentation orale du projet.
-

Livrables attendus

1. **Code source complet**, versionné sur Git.
2. **Projet Unity fonctionnel**.
3. **Analyse UML** (diagrammes de classes, d'activités, d'états et cas d'utilisation).
4. **Tableau Trello** (capture d'écran ou lien).
5. **Diagramme de Gantt**.
6. **Documentation du projet** (PDF ou document Word).
7. **Présentation orale** du projet devant la classe.

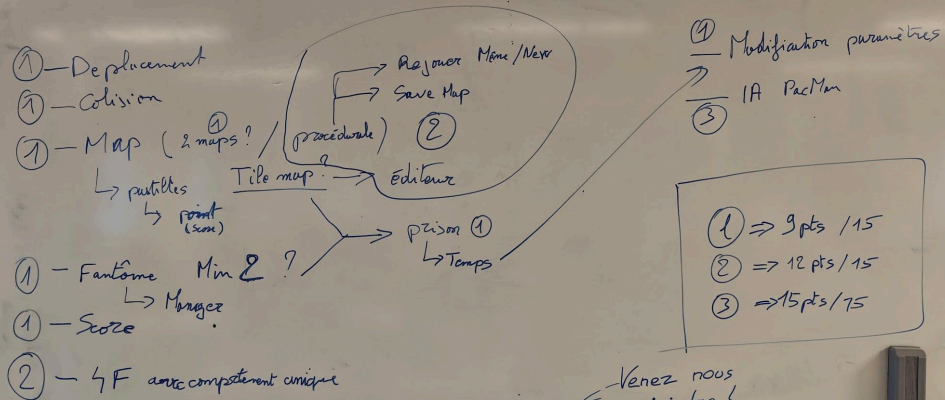
La date du rendu 17/05/2026 23h59 au plus tard. Tout sujet rendu après cette date ne sera pas évalué.

L'évaluation POO sera découpée en trois parties :

- 15 points attribués au projet
- 5 points pour la présentation : 20 minutes (orale + powerpoint + jeu) et 20 minutes de questions

L'évaluation MET aura pour objectif d'évaluer :

- La planification et la répartition des tâches,
- La gestion du contrôle de code source,
- L'analyse UML



Je ne sais pas si je vais m'en servir. Mais ça a une bonne gueule donc je vais voir si je reprends pas des parties du haut sous cette forme.



Critères d'évaluation

Critère	Pondération	Détails
Fonctionnalité du jeu	30%	Le jeu est jouable, fluide et respecte les mécaniques de base de Pac-Man.
Qualité du code C#	25%	Code structuré, orienté objet, commenté et clair.
Analyse UML	20%	Diagrammes complets, cohérents avec le code.
Gestion de projet (Git, Trello, Gantt)	15%	Suivi rigoureux du développement et planification visible.
Présentation et documentation	10%	Clarté, qualité du rendu final, capacité à expliquer les choix.



Extensions possibles (pour les plus avancés)

- Implémentation d'une **IA plus complexe** pour les fantômes (chasse, fuite, regroupement).
- Ajout de **sons** et **animations**.
- Système de **niveaux multiples** ou de **score élevé sauvegardé**.
- Utilisation d'un **Tilemap Editor** pour créer plusieurs labyrinthes.